

```

1  """
2  neuronal_env_physic.py - Environnement vectorisé GPU avec physique réaliste
3  =====
4
5  Changements par rapport à neuronal_env.py :
6
7  Physique
8      - heading séparé de velocity_angle via slip_angle (dérive réaliste)
9      - vitesse max effective = BASE_MAX_SPEED x grip
10     - accélération et freinage modulés par grip et niveau fuel
11     - tout délégué à CarPhysics (envs/physics.py)
12
13     Systèmes
14         - TireSystemGPU → grip, usure selon pluie et composé
15         - FuelSystemGPU → consommation en v², accélération nulle à réservoir
16         - RainSystemGPU → pluie globale, évolution dynamique ou fixe
17
18     Pit stops
19         - pit_tire (action 2) : arrêt n=PIT_TIRE_DURATION steps → pneus neufs
20         - pit_full (action 3) : arrêt N=PIT_FULL_DURATION steps → pneus + refuel
21         - déclenché uniquement si in_pit_zone (géré en amont par
22         pop_manager_physic)
23         - pendant le pit : voiture immobile, invulnérable aux collisions
24
25     Observations : (N, n_rays + 5)
26         distances lidar × n_rays | vitesse | usure | fuel | pluie | composé
27
28     Collision GPU
29         - point-in-polygon vectorisé, zéro transfert CPU
30         - AngularTrack → test dans polygone outer ET hors polygone inner
31         - RectangularTrack → AABB simple
32         - fallback CPU si type inconnu (affiche un warning)
33
34     Interface
35         Identique à neuronal_env.py : reset / step / get_observations /
36         get_render_data
37         → TrainingLoop et circuits_loader n'ont rien à changer côté appel.
38         Nouveaux attributs publics : in_pit_zone (N,) bool, slip_angle (N, 1)
39     """
40
41     import torch
42     import numpy as np
43
44     from physics.physic import CarPhysics, BASE_MAX_SPEED, DT
45     from physics.tires_system import TireSystemGPU, MEDIUM, N_COMPOUNDS
46     from physics.fuel_system import FuelSystemGPU
47     from physics.rain_system import RainSystemGPU

```

```

47 # -----
48 # Constantes propres à l'environnement
49 # -----
50
51 # Durées de pit stop en steps (dt=0.05 → 1 step = 0.05 s)
52 PIT_TIRE_DURATION = 60      # ~3 secondes - changement pneus uniquement
53 PIT_FULL_DURATION = 120     # ~6 secondes - pneus + plein de carburant
54
55 # Rayon autour du spawn considéré comme zone de pit (pixels)
56 PIT_ZONE_RADIUS = 5.0
57
58 # Seuil sigmoid pour déclencher un pit stop (> seuil → pit)
59 PIT_THRESHOLD = 0.5
60
61 # Seuil de proximité aux murs pour la collision par rayons (préfiltrage
62   rapide)
63 RAY_COLLISION_THRESHOLD = 3.0
64
65 class VectorizedCarEnv:
66     """
67     Environnement vectorisé pour N voitures en parallèle sur GPU.
68
69     Paramètres
70     -----
71     spawn_point: (x, y, theta) - position et angle de départ
72     walls: liste de ((x1,y1),(x2,y2)) - segments de murs
73     track: RectangularTrack ou AngularTrack
74     track_width: largeur moyenne du circuit (normalisation)
75     n_cars: taille de la population
76     n_rays: nombre de rayons lidar
77     device: 'cuda' ou 'cpu'
78     collision_threshold: distance min aux murs (préfiltre rayon)
79     rain_mode: 'fixed' | 'dynamic' | 'preset'
80     initial_rain: intensité initiale de la pluie [0, 1]
81     initial_compound: indice du composé de départ (défaut MEDIUM=1)
82     """
83
84     def __init__(
85         self,
86         spawn_point,
87         walls,
88         track,
89         track_width: float = 60.0,
90         n_cars: int = 1000,
91         n_rays: int = 9,
92         device: str = 'cuda',
93         max_speed: float = None,      # ignoré - kept pour compatibilité
94
95 interface

```

```

94     collision_threshold: float = RAY_COLLISION_THRESHOLD,
95     rain_mode: str = 'fixed',
96     initial_rain: float = 0.0,
97     initial_compound: int = MEDIUM,
98 ):
99     self.device = torch.device(device if torch.cuda.is_available() else
'cpu')
100     self.n_cars = n_cars
101     self.n_rays = n_rays
102     self.track_width = track_width
103     self.collision_threshold = collision_threshold
104
105     # -- Spawn -----
106     if len(spawn_point) == 3:
107         self.spawn_x, self.spawn_y, self.spawn_angle = spawn_point
108     else:
109         self.spawn_x, self.spawn_y = spawn_point
110         self.spawn_angle = 0.0
111
112     self.spawn_tensor = torch.tensor(
113         [self.spawn_x, self.spawn_y],
114         dtype=torch.float32, device=self.device
115     )
116
117     # -- Track → tenseurs GPU pour collision -----
118     self.track = track
119     self._setup_track_tensors(track)
120
121     # -- Murs → tenseurs GPU pour raycasting -----
122     starts, ends = [], []
123     for w in walls:
124         starts.append(w[0])
125         ends.append(w[1])
126     self.wall_starts = torch.tensor(starts, dtype=torch.float32,
device=self.device)
127     self.wall_ends = torch.tensor(ends, dtype=torch.float32,
device=self.device)
128
129     # -- Moteur physique -----
130     self.physics = CarPhysics(device=str(self.device))
131
132     # -- Systèmes -----
133     self.tires = TireSystemGPU(n_cars, initial_compound, str(self.device))
134     self.fuel = FuelSystemGPU(n_cars, device=str(self.device))
135     self.rain = RainSystemGPU(

```

```

136         mode=rain_mode, initial_intensity=initial_rain,
device=stor(self.device)
137     )
138
139     # -- Angles des rayons (fixes, calculés une fois) -----
---
140     self.ray_angles = torch.linspace(
141         -np.pi / 2, np.pi / 2, n_rays, device=self.device
142     ) # (n_rays,)
143
144     # -- Tenseurs d'état - initialisés dans reset() -----
--
145     self.pos          = torch.zeros((n_cars, 2), device=self.device)
146     self.speed        = torch.zeros((n_cars, 1), device=self.device)
147     self.heading      = torch.zeros((n_cars, 1), device=self.device)
148     self.slip_angle   = torch.zeros((n_cars, 1), device=self.device)
149     self.alive        = torch.ones(n_cars, dtype=torch.bool,
device=self.device)
150     self.distances    = torch.zeros((n_cars, n_rays), device=self.device)
151
152     # -- État pit stop -----
---
153     self.pit_timer    = torch.zeros(n_cars, dtype=torch.int32,
device=self.device)
154     self.pit_is_full  = torch.zeros(n_cars, dtype=torch.bool,
device=self.device)
155     self.in_pit_zone  = torch.zeros(n_cars, dtype=torch.bool,
device=self.device)
156
157     self.reset()
158
159     # Setup
160     def _setup_track_tensors(self, track):
161         """
162         Convertit la géométrie du track en tenseurs GPU.
163         Les sommets des polygones sont pré-shiftés (roll -1) une seule fois
164         pour éviter de recalculer torch.roll à chaque step.
165         """
166         from tracks.track_geometry import AngularTrack, RectangularTrack
167
168         if isinstance(track, AngularTrack):
169             self.track_type = 'angular'
170
171             outer = torch.tensor(track.outer, dtype=torch.float32,
device=self.device)
172             inner = torch.tensor(track.inner, dtype=torch.float32,
device=self.device)
173
174             # Pré-calcul des sommets suivants (edge i → i+1)
175             self.outer_poly      = outer
176             self.outer_poly_next = torch.roll(outer, -1, dims=0)

```

```

177         self.inner_poly      = inner
178         self.inner_poly_next = torch.roll(inner, -1, dims=0)
179
180     elif isinstance(track, RectangularTrack):
181         self.track_type = 'rectangular'
182         ox_min, ox_max, oy_min, oy_max = track.outer_bounds
183         ix_min, ix_max, iy_min, iy_max = track.inner_bounds
184         self.outer_bounds_t = torch.tensor(
185             [ox_min, ox_max, oy_min, oy_max], dtype=torch.float32,
device=self.device
186         )
187         self.inner_bounds_t = torch.tensor(
188             [ix_min, ix_max, iy_min, iy_max], dtype=torch.float32,
device=self.device
189         )
190
191     else:
192         self.track_type = 'unknown'
193         print(
194             "[neuronal_env_physic] Warning  Type de track inconnu - "
195             "fallback CPU pour la détection de collision (lent)."
196         )
197
198     # Reset
199     def reset(self, randomize_spawn=False, random_range=10.0,
random_angle=0.1):
200         """
201         Remet toutes les voitures au départ.
202
203         Returns : observations Tensor (N, n_rays + 5)
204         """
205         self.pos[:, 0] = self.spawn_x
206         self.pos[:, 1] = self.spawn_y
207
208         if randomize_spawn:
209             self.pos      += torch.randn((self.n_cars, 2), device=self.device)
* random_range
210             self.heading[:] = self.spawn_angle + \
211                 torch.randn((self.n_cars, 1), device=self.device) *
random_angle
212         else:
213             self.heading[:] = self.spawn_angle
214
215         self.speed.zero_()
216         self.slip_angle.zero_()
217         self.alive[:] = True
218         self.distances.zero_()
219
220         self.pit_timer.zero_()
221         self.pit_is_full.zero_()
222         self.in_pit_zone.zero_()

```

```

223
224     self.tires.reset()
225     self.fuel.reset()
226     self.rain.reset()
227
228     return self.get_observations()
229
230     # Step
231     def step(self, actions: torch.Tensor):
232         """
233         Avance d'un pas de temps.
234
235         Args
236         -----
237         actions : Tensor (N, 4)
238            [:, 0] steering   dans [-1, 1]
239            [:, 1] throttle   dans [-1, 1]   (négatif = frein)
240            [:, 2] pit_tire   dans [0, 1]    (> PIT_THRESHOLD → pit pneus)
241            [:, 3] pit_full   dans [0, 1]    (> PIT_THRESHOLD → pit complet)
242
243         Returns
244         -----
245         observations : Tensor (N, n_rays + 5)
246         rewards      : Tensor (N,)
247         done         : Tensor (N,) bool
248         """
249         steering = actions[:, 0:1] # (N, 1)
250         throttle = actions[:, 1:2] # (N, 1)
251         sig_tire = actions[:, 2]    # (N,)
252         sig_full = actions[:, 3]    # (N,)
253
254         # -- 1. Gestion pit stop -----
255
256         in_pit = self.pit_timer > 0 # (N,) - voitures actuellement au stand
257
258         # Décrémenter le timer des voitures au stand
259         self.pit_timer = (self.pit_timer - 1).clamp(min=0)
260
261         # Appliquer le pit stop quand le timer vient d'atteindre 0
262         just_finished = in_pit & (self.pit_timer == 0) # (N,)
263         if just_finished.any():
264             self.tires.apply_pit_stop(just_finished)
265             full_done = just_finished & self.pit_is_full
266             if full_done.any():
267                 self.fuel.refuel(full_done)
268                 self.pit_is_full[just_finished] = False
269
270         # Déclencher un nouveau pit stop
271         # Condition : vivante + pas déjà au stand + signal > seuil
272         can_pit = self.alive & ~in_pit # (N,)

```

```

273     new_tire      = can_pit & (sig_tire > PIT_THRESHOLD)
274     new_full      = can_pit & (sig_full > PIT_THRESHOLD)
275     new_tire      = new_tire & ~new_full           # pit_full prioritaire
276
277     if new_tire.any():
278         self.pit_timer[new_tire] = PIT_TIRE_DURATION
279         self.pit_is_full[new_tire] = False
280
281     if new_full.any():
282         self.pit_timer[new_full] = PIT_FULL_DURATION
283         self.pit_is_full[new_full] = True
284
285     # -- 2. Physique -----
286
287     # Voitures actives = vivantes ET pas au stand
288     active = self.alive & ~in_pit    # (N,)
289
290     rain = self.rain.intensity           # float
291     scalaire
292     grip = self.tires.grip_multiplier(rain)           # (N, 1)
293     effective_throttle = throttle * self.fuel.fuel_throttle_factor() #
294     (N, 1)
295
296     self.pos, self.speed, self.heading, self.slip_angle =
297     self.physics.step(
298         self.pos, self.speed, self.heading, self.slip_angle,
299         active, steering, effective_throttle, grip
300     )
301
302     # -- 3. Mise à jour des systèmes -----
303
304     self.tires.update_wear(self.speed, rain)
305     self.fuel.consume(self.speed)
306     self.rain.step()
307
308     # -- 4. Zone de pit -----
309
310     dist_to_spawn = torch.norm(
311         self.pos - self.spawn_tensor.unsqueeze(0), dim=1
312     ) # (N,)
313     self.in_pit_zone = dist_to_spawn < PIT_ZONE_RADIUS # (N,) bool
314
315     # -- 5. Raycasting -----
316
317     self.distances = self._compute_lidar()
318
319     # -- 6. Détection de collision -----
320
321     ray_crash = (self.distances < self.collusion_threshold).any(dim=1)
322     # (N,)

```

```

315         track_crash = self._check_track_collision_gpu()
316         # (N,)
317
318         # Les voitures au stand sont invulnérables
319         crashes = (ray_crash | track_crash) & ~in_pit
320         self.alive = self.alive & ~crashes
321
322         # -- 7. Observations et retour -----
323
324         observations = self.get_observations()
325         rewards      = self.speed.squeeze(1) * self.alive.float()
326         dones         = ~self.alive
327
328         return observations, rewards, dones
329
330     # Raycasting
331     def _compute_lidar(self) → torch.Tensor:
332         """
333         Calcule les distances lidar pour toutes les voitures.
334
335         heading (N, 1) + ray_angles (n_rays,) → broadcast (N, n_rays)
336         Puis aplatissage → (N×n_rays, 2) pour compute_ray_intersections.
337
338         Returns : (N, n_rays) distances en pixels
339         """
340         from learnings.ray_casting.gpu_raycasting import
341         compute_ray_intersections
342
343         global_angles = self.heading + self.ray_angles # (N, n_rays) broadcast
344         flat_angles = global_angles.view(-1) # (N×n_rays,)
345
346         ray_dirs = torch.stack(
347             [torch.cos(flat_angles), torch.sin(flat_angles)], dim=1
348         ) # (N×n_rays, 2)
349         ray_origins = self.pos.repeat_interleave(self.n_rays, dim=0) #
350         (N×n_rays, 2)
351
352         dists = compute_ray_intersections(
353             ray_origins, ray_dirs,
354             self.wall_starts, self.wall_ends,
355             max_dist=3.0 * self.track_width
356         ) # (N×n_rays,)
357
358         return dists.view(self.n_cars, self.n_rays) # (N, n_rays)
359
360     # Collision GPU
361     def _inside_polygon_gpu(
362         self,
363         points      : torch.Tensor, # (N, 2)
364         poly        : torch.Tensor, # (M, 2)
365         poly_next   : torch.Tensor, # (M, 2) = roll(poly, -1) pré-calculé

```



```

362     ) → torch.Tensor:
363         """
364         Test point-in-polygon par ray casting, 100% GPU.
365
366         Pour chaque point, on lance un rayon horizontal vers +x et on compte
367         le nombre d'arêtes du polygone qu'il traverse.
368         Impair → intérieur, pair → extérieur.
369
370         Complexité mémoire :  $O(N \times M)$  - pour  $N=1000$ ,  $M=200 \rightarrow \sim 3$  MB. ✓
371
372         Returns : (N,) bool - True si le point est à l'intérieur
373         """
374         # Coordonnées des points : (N, 1)
375         px = points[:, 0].unsqueeze(1) # (N, 1)
376         py = points[:, 1].unsqueeze(1) # (N, 1)
377
378         # Coordonnées des arêtes : (1, M)
379         x1 = poly[:, 0].unsqueeze(0) # (1, M)
380         y1 = poly[:, 1].unsqueeze(0) # (1, M)
381         x2 = poly_next[:, 0].unsqueeze(0) # (1, M)
382         y2 = poly_next[:, 1].unsqueeze(0) # (1, M)
383
384         # Condition 1 : py est strictement entre y1 et y2
385         cond1 = (y1 > py) != (y2 > py) # (N, M) bool
386
387         # Abscisse de l'intersection du rayon horizontal y=py avec l'arête
388         dy = y2 - y1 # (1, M)
389         safe_dy = torch.where(dy.abs() < 1e-9, torch.full_like(dy, 1e-9), dy)
390         xinters = x1 + (py - y1) * (x2 - x1) / safe_dy # (N, M)
391
392         # Condition 2 : l'intersection est à droite de px
393         cond2 = px < xinters # (N, M) bool
394
395         # Nombre de traversées - impair = intérieur
396         crossings = (cond1 & cond2).int().sum(dim=1) # (N,)
397         return (crossings % 2) == 1 # (N,) bool
398
399     def _check_track_collision_gpu(self) → torch.Tensor:
400         """
401         Retourne un masque (N,) bool : True = voiture hors circuit.
402         Zéro transfert GPU→CPU.
403         """
404         if self.track_type == 'angular':
405             in_outer = self._inside_polygon_gpu(
406                 self.pos, self.outer_poly, self.outer_poly_next
407             )
408             in_inner = self._inside_polygon_gpu(
409                 self.pos, self.inner_poly, self.inner_poly_next
410             )
411             # Sur le circuit = dans outer ET hors inner
412             return ~(in_outer & ~in_inner) # (N,) bool

```

```

413
414         elif self.track_type == 'rectangular':
415             x = self.pos[:, 0]
416             y = self.pos[:, 1]
417             ox_min, ox_max, oy_min, oy_max = self.outer_bounds_t
418             ix_min, ix_max, iy_min, iy_max = self.inner_bounds_t
419             in_outer = (x ≥ ox_min) & (x ≤ ox_max) & (y ≥ oy_min) & (y ≤
oy_max)
420             in_inner = (x ≥ ix_min) & (x ≤ ix_max) & (y ≥ iy_min) & (y ≤
iy_max)
421             return ~(in_outer & ~in_inner)
422
423         else:
424             # Fallback CPU - affiché au __init__, ne devrait pas arriver
425             collisions = torch.zeros(self.n_cars, dtype=torch.bool,
device=self.device)
426             pos_cpu = self.pos.cpu().numpy()
427             for i in range(self.n_cars):
428                 if self.alive[i] and not self.track.is_inside(*pos_cpu[i]):
429                     collisions[i] = True
430             return collisions
431
432     # Observations
433     def get_observations(self) → torch.Tensor:
434         """
435         Construit le vecteur d'observation pour toutes les voitures.
436
437         Returns : Tensor (N, n_rays + 5)
438             [0 : n_rays]      distances lidar normalisées [0, 1]
439             [n_rays]          vitesse normalisée [0, 1]
440             [n_rays + 1]      usure pneu [0, 1]
441             [n_rays + 2]      niveau carburant [0, 1]
442             [n_rays + 3]      intensité pluie [0, 1]
443             [n_rays + 4]      composé normalisé [0, 1]
444         """
445         max_range = 3.0 * self.track_width
446
447         rays = (self.distances / max_range).clamp(0.0, 1.0)
448         # (N, n_rays)
449         speed = (self.speed / BASE_MAX_SPEED).clamp(0.0, 1.0)
450         # (N, 1)
451         wear = self.tires.wear.unsqueeze(1)
452         # (N, 1)
453         fuel = self.fuel.fuel.unsqueeze(1)
454         # (N, 1)
455         rain = torch.full(
456             (self.n_cars, 1), self.rain.intensity, device=self.device
457         )
458         # (N, 1)
459         compound = (self.tires.compound.float() / N_COMPOUNDS).unsqueeze(1)
460         # (N, 1)

```

```

455
456         return torch.cat([rays, speed, wear, fuel, rain, compound], dim=1)
457     # (N, n_rays+5)
458
459     # Render
460     def get_render_data(self) → dict:
461         """
462         Renvoie les données CPU pour l'affichage PyGame.
463
464         Clés ajoutées par rapport à v1 :
465         slip_angle : pour colorer les voitures en dérive
466         in_pit_zone : pour afficher la zone de pit
467         pit_timer : pour afficher le compte à rebours au stand
468         """
469         return {
470             'pos' : self.pos.cpu().numpy(),
471             'angle' : self.heading.cpu().numpy(),
472             'alive' : self.alive.cpu().numpy(),
473             'speed' : self.speed.cpu().numpy(),
474             'slip_angle' : self.slip_angle.cpu().numpy(),
475             'in_pit_zone' : self.in_pit_zone.cpu().numpy(),
476             'pit_timer' : self.pit_timer.cpu().numpy(),
477         }
478
479     # Utilitaires
480     def get_alive_count(self) → int:
481         return self.alive.sum().item()
482
483     def is_all_dead(self) → bool:
484         return not self.alive.any()
485
486
487
488
489
490
491     """
492     training_file.py – Entraînement AG avec physique réaliste
493     =====
494
495     Coordonne :
496     - neuronal_env_physic (physique réaliste, pneus, carburant, pluie)
497     - neural_network_physic (4 sorties, entrées étendues)
498     - pop_manager_physic (curriculum learning phases 1 & 2)
499     - fitness_tracker (inchangé)
500     - circuits_loader_physic (instancie neuronal_env_physic)
501
502     Usage :
503     # Entraînement basique (piste sèche, Medium)

```

```

504     python -m test.test_physic --generations 100 --population 100 --
frequency_showgen 0 --random_train 0 --circuit speed_ring_gt --nb_laps 1 --
nb_steps 2000 --phase2_gen 2000

505
506     # Pluie dynamique, démarrage en Wet, phase 2 à la génération 20
507     python -m test.test_genetic_physic --generations 100 --rain_mode dynamic -
-initial_rain 0.5 --compound wet --phase2_gen 20

508
509     # Phase 2 débloquée par seuil de fitness plutôt que par génération
510     python -m test.test_genetic_physic --generations 200 --phase2_gen -1 --
phase2_fitness 5000

511
512     # Multi-circuit (les 2 plus faciles), affichage toutes les 5 générations
513     python -m test.test_genetic_physic --random_train 2 --frequency_showgen 5

514
515     # Reprendre un checkpoint
516     python -m test.test_genetic_physic --checkpoint
checkpoints/nascar_.../gen_50.pt
517     """
518
519     import os
520     os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
521     import torch
522     import argparse
523     from datetime import datetime
524     from learnings.genetic_algorithm.fitness_tracker import FitnessTracker
525     from learnings.genetic_algorithm.pop_manager_physic import PopulationManager,
TrainingLoop
526     from tracks.circuits_loader import load_circuits
527     from physics.tires_system import (
528         COMPOUNDS, HARD, MEDIUM, SOFT, WET, HEAVY_WET
529     )
530
531     # Mapping nom → indice pour --compound
532     _COMPOUND_MAP = {
533         "hard" : HARD,
534         "medium" : MEDIUM,
535         "soft" : SOFT,
536         "wet" : WET,
537         "heavywet" : HEAVY_WET,
538     }
539
540
541     # -----
542     # Arguments
543     # -----
544
545     def parse_args():
546         parser = argparse.ArgumentParser(

```

```

547         description="Entraînement AG v2 – physique réaliste (pneus / carburant
/ pluie)"
548     )
549
550     # -- Entraînement -----
551     --
552     parser.add_argument('--generations', type=int, default=100)
553     parser.add_argument('--population', type=int, default=1000)
554     parser.add_argument('--n_rays', type=int, default=9)
555     parser.add_argument('--save_every', type=int, default=10,
help="Sauvegarder tous les N générations")
556     parser.add_argument('--device', type=str, default='cuda', choices=['cuda',
'cpu'])
557     parser.add_argument('--checkpoint', type=str, default=None, help="Chemin
vers un checkpoint à reprendre")
558     parser.add_argument('--frequency_showgen', type=int, default=-1,
help="Afficher le rendu toutes les N générations (-1 = jamais)")
559
560     # -- Circuits -----
561     --
562     parser.add_argument('--random_train', type=int, default=1, help="Nombre de
circuits (triés par difficulté). 0 = utiliser --circuit")
563     parser.add_argument('--circuit', type=str, nargs='+', default=['nascar'],
choices=['nascar', 'rectangle', 'speed_ring_gt', 'w_track'], help="Circuit(s)
à utiliser si --random_train 0")
564     parser.add_argument('--nb_laps', type=int, default=1, help="Tours pour
terminer une génération (prioritaire sur nb_steps)")
565     parser.add_argument('--nb_steps', type=int, default=2000, help="Steps max
par génération (fallback si nb_laps non atteint)")
566
567     # -- Mutation -----
568     --
569     parser.add_argument('--mutation_start', type=float, default=0.3)
570     parser.add_argument('--mutation_end', type=float, default=0.05)
571     parser.add_argument('--mutation_decay', type=float, default=0.975)
572
573     # -- Systèmes physiques -----
574     parser.add_argument('--rain_mode', type=str, default='fixed', choices=
['fixed', 'dynamic', 'preset'], help="Mode d'évolution de la pluie")
575     parser.add_argument('--initial_rain', type=float, default=0.0,
help="Intensité de pluie initiale [0.0=sec, 1.0=déluge]")
576     parser.add_argument('--compound', type=str, default='medium',
choices=list(_COMPOUND_MAP.keys()), help="Composé de départ pour tous les
agents")
577
578     # -- Curriculum learning (savoir quand on accède aux nouvelles actions) --
579     --
580     parser.add_argument('--phase2_gen', type=int, default=10, help="Génération
de déclenchement de la phase 2 (-1 = désactivé)")
581     parser.add_argument('--phase2_fitness', type=float, default=None,
help="Seuil avg_fitness pour déclencher la phase 2 (None = désactivé)")

```

```
578
579     return parser.parse_args()
580
581
582 # -----
583 # Main
584 # -----
585
586 def main():
587     args = parse_args()
588
589     # -- 1. GPU -----
590     --
591     if args.device == 'cuda' and not torch.cuda.is_available():
592         print("CUDA non disponible, passage en CPU")
593         args.device = 'cpu'
594
595     if args.device == 'cuda':
596         print(f"GPU : {torch.cuda.get_device_name(0)}")
597         print(f"VRAM : {torch.cuda.get_device_properties(0).total_memory /
598 1e9:.1f} GB")
599
600     initial_compound = _COMPOUND_MAP[args.compound]
601     phase2_gen = args.phase2_gen if args.phase2_gen ≥ 0 else None
602
603     print(
604         f"\nSystèmes : pluie={args.rain_mode} (init={args.initial_rain:.2f}) |
605 "
606         f"composé={args.compound.capitalize()}"
607     )
608     print(
609         f"Curriculum : phase 2 à gen≥{phase2_gen} "
610         f"ou avg_fitness≥{args.phase2_fitness}\n"
611     )
612
613     # -- 2. Circuits -----
614     --
615     configs = load_circuits(
616         n = args.random_train if args.random_train > 0 else None,
617         names = args.circuit if args.random_train == 0 else None,
618         n_cars = args.population,
619         n_rays = args.n_rays,
620         device = args.device,
621         rain_mode = args.rain_mode,
622         initial_rain = args.initial_rain,
623         initial_compound = initial_compound,
624     )
625
626     if len(configs) == 1:
```

```

623         cfg = configs[0]
624         print(f"Circuit unique : {cfg['name']} (difficulté
{cfg['difficulty']})")
625     else:
626         print(f"{len(configs)} circuits chargés :")
627         for c in configs:
628             print(f" - {c['name']} (difficulté {c['difficulty']})")
629
630     cfg = configs[0]
631     env = cfg["env"]
632     checkpoints = cfg["checkpoints"]
633     walls = cfg["walls"]
634
635     print(f"Voitures : {args.population} | Rayons : {args.n_rays} |
Checkpoints : {len(checkpoints)}")
636
637     # -- 3. FitnessTracker -----
638     -
639     fitness_tracker = FitnessTracker(
640         checkpoints = checkpoints,
641         spawn_point = (env.spawn_x, env.spawn_y, env.spawn_angle),
642         n_cars = args.population,
643         track_width = env.track_width,
644         walls = walls,
645         device = args.device,
646     )
647
648     # -- 4. PopulationManager -----
649     -
650     population_manager = PopulationManager(
651         n_population = args.population,
652         n_rays = args.n_rays,
653         initial_mutation_rate = args.mutation_start,
654         final_mutation_rate = args.mutation_end,
655         mutation_decay = args.mutation_decay,
656         device = args.device,
657     )
658
659     # -- 5. Checkpoint -----
660     --
661     if args.checkpoint:
662         print(f"\nChargement checkpoint : {args.checkpoint}")
663         population_manager.load_population_from_file(args.checkpoint)
664
665     # -- 6. TrainingLoop -----
666     --
667     training_loop = TrainingLoop(
668         env = env,
669         population_manager = population_manager,
670         fitness_tracker = fitness_tracker,
671         frequency_show = args.frequency_showgen,

```

```

668         walls = walls,
669         all_configs = configs,
670         max_laps = args.nb_laps,
671         max_steps = args.nb_steps,
672         phase2_gen = phase2_gen,
673         phase2_fitness = args.phase2_fitness,
674     )
675
676     # -- 7. Dossier de sauvegarde -----
677
678     circuit_names = "_".join(c["name"] for c in configs)
679     timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
680     save_dir = f"checkpoints/{circuit_names}_{timestamp}"
681     os.makedirs(save_dir, exist_ok=True)
682
683     # Sauvegarder la config lisiblement
684     config_path = os.path.join(save_dir, "config.txt")
685     with open(config_path, 'w') as f:
686         f.write(f"Circuits: {circuit_names}\n")
687         f.write(f"Population: {args.population}\n")
688         f.write(f"Générations: {args.generations}\n")
689         f.write(f"Rayons: {args.n_rays}\n")
690         f.write(f"Mutation: {args.mutation_start} → {args.mutation_end}
691 (decay={args.mutation_decay})\n")
692         f.write(f"Device: {args.device}\n")
693         f.write(f"Pluie: {args.rain_mode} (init={args.initial_rain})\n")
694         f.write(f"Composé: {args.compound}\n")
695         f.write(f"Phase 2 gen: {phase2_gen}\n")
696         f.write(f"Phase 2 fitness: {args.phase2_fitness}\n")
697
698     print(f"\nSauvegardes dans : {save_dir}")
699
700     # -- 8. Entraînement -----
701
702     print(f"\nDÉBUT DE L'ENTRAÎNEMENT")
703     print("=" * 100 + "\n")
704
705     training_loop.train(
706         n_generations = args.generations,
707         save_every = args.save_every,
708         save_path = save_dir,
709     )
710
711     # -- 9. Statistiques finales -----
712
713     print(f"\n{'='*100}")
714     print("ENTRAÎNEMENT TERMINÉ")
715     print(f"{'='*100}")
716     print(f"Fichiers sauvegardés dans : {save_dir}")

```



```

714     do_visu = input("\nAfficher les courbes de progression ? (Y/N) :
715 ").strip().upper()
716     if do_visu == "Y":
717         from learnings.genetic_algorithm.show_improve import main as show_main
718         show_main(save_dir)
719
720 if __name__ == "__main__":
721     main()
722
723
724
725
726
727
728 """
729 neural_network_physic.py – Réseau de neurones avec physique réaliste
730 =====
731
732 Changements par rapport à neural_network.py :
733
734 Entrées : n_rays + 1 → n_rays + 1 + n_system_obs
735 Nouvelles observations systèmes (par défaut 4) :
736     - usure pneu      [0, 1]
737     - niveau fuel     [0, 1]
738     - intensité pluie [0, 1]
739     - composé actuel  [0, 1] (normalisé : indice / N_COMPOUNDS)
740
741 Sorties : 2 → 4
742     - steering : tanh → [-1, 1] (inchangé)
743     - throttle : tanh → [-1, 1] (sigmoid → tanh pour autoriser le
744 freinage)
745     - pit_tire  : sigmoid → [0, 1] (> 0.5 + in_pit_zone → pit pneus
746 seulement)
747     - pit_full  : sigmoid → [0, 1] (> 0.5 + in_pit_zone → pit complet)
748
749 Taille du génome (n_rays=9, n_system_obs=4, h1=32, h2=16) :
750     W1 : 32 × 14 = 448    b1 : 32
751     W2 : 16 × 32 = 512    b2 : 16
752     W3 : 4 × 16 = 64      b3 : 4
753     Total = 1076
754
755 Note sur les hidden sizes :
756     h1 et h2 ont été augmentés (16→32, 8→16) pour compenser
757     l'augmentation du nombre d'entrées et de sorties.
758     Si les temps d'entraînement sont trop longs, on peut revenir à 16/8.
759
760 Compatibilité :
761     pop_manager_physic.py doit importer depuis neural_network_physic
762     Les checkpoints de neural_network.py sont INCOMPATIBLES (taille génome
différente)

```

```

761
762 Usage :
763     from learnings.genetic_algorithm.neural_network_physic import (
764         NeuralAgent, VectorizedNeuralPopulation
765     )
766     """
767
768     import torch
769     import torch.nn as nn
770
771     # Nombre d'observations système ajoutées aux rayons + vitesse
772     # (wear, fuel, rain, compound) → 4
773     N_SYSTEM_OBS = 4
774
775     # Nombre d'actions de sortie
776     # (steering, throttle, pit_tire, pit_full) → 4
777     N_ACTIONS = 4
778
779
780     class NeuralAgent(nn.Module):
781         """
782         Réseau MLP pour contrôler une voiture avec gestion des systèmes.
783
784         Architecture :
785             Input : n_rays + 1 (vitesse) + n_system_obs
786             Hidden1: h1 neurones (ReLU)
787             Hidden2: h2 neurones (ReLU)
788             Output : 4 neurones
789                 [0] steering : tanh → [-1, 1]
790                 [1] throttle : tanh → [-1, 1] (négatif = freinage)
791                 [2] pit_tire : sigmoid → [0, 1]
792                 [3] pit_full : sigmoid → [0, 1]
793         """
794
795         def __init__(
796             self,
797             n_rays : int = 9,
798             n_system_obs : int = N_SYSTEM_OBS,
799             hidden_size1 : int = 32,
800             hidden_size2 : int = 16,
801             device : str = 'cuda'
802         ):
803             super(NeuralAgent, self).__init__()
804
805             self.n_rays = n_rays
806             self.n_system_obs = n_system_obs
807             self.device = device
808
809             input_size = n_rays + 1 + n_system_obs # rayons + vitesse + systèmes
810
811             self.network = nn.Sequential(

```

```

812         nn.Linear(input_size, hidden_size1),
813         nn.ReLU(),
814         nn.Linear(hidden_size1, hidden_size2),
815         nn.ReLU(),
816         nn.Linear(hidden_size2, N_ACTIONS), # 4 sorties
817     )
818
819     self._initialize_weights()
820
821     def _initialize_weights(self):
822         for layer in self.network:
823             if isinstance(layer, nn.Linear):
824                 nn.init.xavier_uniform_(layer.weight)
825                 nn.init.zeros_(layer.bias)
826
827     def forward(self, observations: torch.Tensor) → torch.Tensor:
828         """
829         Propagation avant.
830
831         Args:
832             observations : Tensor (batch, n_rays + 1 + n_system_obs)
833                [:, :n_rays]         distances lidar normalisées [0, 1]
834                [:, n_rays]         vitesse normalisée [0, 1]
835                [:, n_rays+1]       usure pneu [0, 1]
836                [:, n_rays+2]       niveau fuel [0, 1]
837                [:, n_rays+3]       intensité pluie [0, 1]
838                [:, n_rays+4]       composé normalisé [0, 1]
839
840         Returns:
841             actions : Tensor (batch, 4)
842                [:, 0] steering ∈ [-1, 1]
843                [:, 1] throttle ∈ [-1, 1]
844                [:, 2] pit_tire ∈ [0, 1]
845                [:, 3] pit_full ∈ [0, 1]
846         """
847         raw = self.network(observations) # (batch, 4)
848
849         steering = torch.tanh(raw[:, 0:1]) # [-1, 1]
850         throttle = torch.tanh(raw[:, 1:2]) # [-1, 1] ←
851         # freinage possible
852         pit_tire = torch.sigmoid(raw[:, 2:3]) # [0, 1]
853         pit_full = torch.sigmoid(raw[:, 3:4]) # [0, 1]
854
855         return torch.cat([steering, throttle, pit_tire, pit_full], dim=1) #
856         (batch, 4)
857
858         # -----
859         --
860
861         # Interface génome – identique à l'ancienne
862         def get_genome(self) → torch.Tensor:
863             """Extrait tous les poids sous forme de vecteur 1D."""

```

```

860         return torch.cat([p.data.view(-1) for p in self.parameters()])
861
862     def set_genome(self, genome: torch.Tensor):
863         """Injecte un génome (vecteur 1D) dans le réseau."""
864         offset = 0
865         for param in self.parameters():
866             numel = param.numel()
867             param.data = genome[offset:offset + numel].view(param.shape)
868             offset += numel
869
870     def mutate(self, mutation_rate: float = 0.3, mutation_strength: float =
0.1):
871         """Mutation aléatoire uniforme des poids."""
872         with torch.no_grad():
873             for param in self.parameters():
874                 mask = torch.rand_like(param) < mutation_rate
875                 noise = torch.rand_like(param) * 2 * mutation_strength -
mutation_strength
876                 param.data += noise * mask.float()
877
878     def clone(self) → 'NeuralAgent':
879         c = NeuralAgent(
880             n_rays=self.n_rays, n_system_obs=self.n_system_obs,
device=self.device
881         ).to(self.device)
882         c.load_state_dict(self.state_dict())
883         return c
884
885     def crossover(self, other: 'NeuralAgent', crossover_rate: float = 0.5) →
'NeuralAgent':
886         child = NeuralAgent(
887             n_rays=self.n_rays, n_system_obs=self.n_system_obs,
device=self.device
888         ).to(self.device)
889         g1 = self.get_genome()
890         g2 = other.get_genome()
891         mask = torch.rand_like(g1) < crossover_rate
892         child.set_genome(torch.where(mask, g1, g2))
893         return child
894
895     def save_to_file(self, filepath: str):
896         torch.save(self.state_dict(), filepath)
897
898     def load_from_file(self, filepath: str):
899         self.load_state_dict(torch.load(filepath))
900
901
902 class VectorizedNeuralPopulation:
903     """
904     Population entière gérée en un seul forward GPU via bmm.
905

```

```

906 Identique à v1 sauf :
907     - input_size = n_rays + 1 + n_system_obs
908     - output_size = 4 (au lieu de 2)
909     - h1=32, h2=16 (au lieu de 16, 8)
910     - throttle → tanh, pit_tire / pit_full → sigmoid
911     """
912
913     def __init__(
914         self,
915         n_agents      : int = 1000,
916         n_rays        : int = 9,
917         n_system_obs  : int = N_SYSTEM_OBS,
918         hidden_size1  : int = 32,
919         hidden_size2  : int = 16,
920         device        : str = 'cuda'
921     ):
922         self.n_agents      = n_agents
923         self.n_rays        = n_rays
924         self.n_system_obs  = n_system_obs
925         self.h1            = hidden_size1
926         self.h2            = hidden_size2
927         self.device        = torch.device(device if torch.cuda.is_available()
928     else 'cpu')
929
930         # Taille du génome calculée depuis un modèle de référence
931         ref = NeuralAgent(n_rays, n_system_obs, hidden_size1,
932             hidden_size2).to(self.device)
933         self.genome_size = sum(p.numel() for p in ref.parameters())
934         self.input_size = n_rays + 1 + n_system_obs
935
936         # Génomes initiaux (distribution gaussienne centrée, std=0.5)
937         self.genomes = torch.randn(
938             n_agents, self.genome_size, device=self.device
939         ) * 0.5
940
941     def forward_vectorized(self, observations: torch.Tensor) → torch.Tensor:
942         """
943         Forward pass pour toute la population en une seule passe GPU.
944
945         Args:
946             observations : Tensor (N, input_size)
947
948         Returns:
949             actions : Tensor (N, 4)
950                [:, 0] steering ∈ [-1, 1]
951                [:, 1] throttle ∈ [-1, 1]
952                [:, 2] pit_tire ∈ [0, 1]
953                [:, 3] pit_full ∈ [0, 1]
954
955         Dimensions des tenseurs de poids (bmm = batch matrix multiply) :
956             W1 (N, h1, input) × x (N, input, 1) → (N, h1, 1)

```

```

955         W2 (N, h2, h1)      × x (N, h1, 1) → (N, h2, 1)
956         W3 (N, 4, h2)      × x (N, h2, 1) → (N, 4, 1)
957         """
958         N = self.n_agents
959         i = self.input_size
960         h1 = self.h1
961         h2 = self.h2
962
963         # -- Découpage des génomes en matrices de poids -----
964         --
965         offset = 0
966         W1 = self.genomes[:, offset:offset + h1*i ].view(N, h1, i ); offset
967 += h1 * i
968         b1 = self.genomes[:, offset:offset + h1 ].view(N, h1, 1 ); offset
969 += h1
970         W2 = self.genomes[:, offset:offset + h2*h1].view(N, h2, h1); offset
971 += h2 * h1
972         b2 = self.genomes[:, offset:offset + h2 ].view(N, h2, 1 ); offset
973 += h2
974         W3 = self.genomes[:, offset:offset + 4*h2 ].view(N, 4, h2); offset
975 += 4 * h2
976         b3 = self.genomes[:, offset:offset + 4 ].view(N, 4, 1 )
977
978         # -- Forward pass vectorisé -----
979         --
980         x = observations.unsqueeze(-1)           # (N, input, 1)
981         x = torch.relu(torch.bmm(W1, x) + b1)     # (N, h1, 1)
982         x = torch.relu(torch.bmm(W2, x) + b2)     # (N, h2, 1)
983         x = torch.bmm(W3, x) + b3                # (N, 4, 1)
984         x = x.squeeze(-1)                       # (N, 4)
985
986         # -- Activations par sortie -----
987         --
988         steering = torch.tanh(x[:, 0:1])          # (N, 1) [-1, 1]
989         throttle = torch.tanh(x[:, 1:2])          # (N, 1) [-1, 1]
990         pit_tire = torch.sigmoid(x[:, 2:3])        # (N, 1) [0, 1]
991         pit_full = torch.sigmoid(x[:, 3:4])        # (N, 1) [0, 1]
992
993         return torch.cat([steering, throttle, pit_tire, pit_full], dim=1) #
994 (N, 4)
995
996         # Interface population – identique à v1
997         def get_agent(self, index: int) → NeuralAgent:
998             agent = NeuralAgent(
999                 self.n_rays, self.n_system_obs, self.h1, self.h2
1000             ).to(self.device)
1001             agent.set_genome(self.genomes[index])
1002             return agent
1003
1004         def set_agent(self, index: int, agent: NeuralAgent):

```

```

997         self.genomes[index] = agent.get_genome()
998
999         def mutate_all(self, mutation_rate: float = 0.1, mutation_strength: float
= 0.3):
1000             mask = torch.rand_like(self.genomes) < mutation_rate
1001             noise = (torch.rand_like(self.genomes) * 2.0 - 1.0) *
mutation_strength
1002             self.genomes += noise * mask.float()
1003
1004         def save_population(self, filepath: str):
1005             torch.save({
1006                 'genomes'      : self.genomes,
1007                 'n_agents'     : self.n_agents,
1008                 'n_rays'       : self.n_rays,
1009                 'n_system_obs' : self.n_system_obs,
1010                 'h1'           : self.h1,
1011                 'h2'           : self.h2,
1012             }, filepath)
1013
1014         def load_population(self, filepath: str):
1015             data = torch.load(filepath, map_location=self.device)
1016             self.genomes      = data['genomes'].to(self.device)
1017             self.n_agents     = data['n_agents']
1018             self.n_rays       = data['n_rays']
1019             self.n_system_obs = data.get('n_system_obs', N_SYSTEM_OBS)
1020             self.h1           = data.get('h1', 32)
1021             self.h2           = data.get('h2', 16)
1022
1023
1024

```

"""

pop_manager_physic.py - Gestionnaire de population avec curriculum learning

Changements par rapport à pop_manager.py :

1. Import depuis neural_network_physic
→ VectorizedNeuralPopulation avec input_size = n_rays + 1 + n_system_obs
→ 4 sorties au lieu de 2 (steering, throttle, pit_tire, pit_full)
2. PopulationManager accepte n_system_obs, hidden_size1, hidden_size2
3. TrainingLoop - Curriculum Learning en 2 phases :
 - Phase 1 - Conduite pure
actions[:, 2:4] = 0.0 → pit_tire et pit_full masqués (toujours < seuil 0.5)
L'agent apprend à conduire sans se préoccuper des pit stops.
 - Phase 2 - Stratégie pit stop débloquée

```

1043         Déclenchée à la génération `phase2_gen` OU quand avg_fitness ≥
`phase2_fitness`
1044         (le premier critère atteint gagne).
1045         actions[:, 2:4] actives UNIQUEMENT pour les voitures dans la zone de
pit
1046         (env.in_pit_zone). Hors zone → masqué à 0.0.
1047
1048         Le réseau a toujours 4 sorties dès le départ - génome compatible entre
les deux phases.
1049         Transition instantanée, sans rechargement ni modification d'architecture.
1050
1051 Usage :
1052     from learnings.genetic_algorithm.pop_manager_physic import
PopulationManager, TrainingLoop
1053     """
1054
1055     import os
1056     import torch
1057     from learnings.genetic_algorithm.neural_network_physic import (
1058         NeuralAgent,
1059         VectorizedNeuralPopulation,
1060         N_SYSTEM_OBS,
1061     )
1062     from render.render_GPU import VectorizedRenderer
1063
1064     # Seuil de décision pour les actions pit (sigmoid > PIT_THRESHOLD → pit
déclenché)
1065     PIT_THRESHOLD = 0.5
1066
1067     class PopulationManager:
1068         """
1069         Gère la population, la sélection, le crossover et la mutation.
1070         Identique à v1 sauf les paramètres de construction de
VectorizedNeuralPopulation.
1071         """
1072
1073         def __init__(
1074             self,
1075             n_population: int = 1000,
1076             n_rays: int = 9,
1077             n_system_obs: int = N_SYSTEM_OBS,
1078             hidden_size1: int = 32,
1079             hidden_size2: int = 16,
1080             initial_mutation_rate: float = 0.3,
1081             final_mutation_rate: float = 0.001,
1082             mutation_decay: float = 0.995,
1083             mutation_strength: float = 0.3,
1084             device: str = 'cuda',
1085         ):
1086             self.n_population = n_population
1087             self.n_rays = n_rays

```



```

1088     self.n_system_obs = n_system_obs
1089     self.hidden_size1 = hidden_size1
1090     self.hidden_size2 = hidden_size2
1091     self.device = torch.device(device if torch.cuda.is_available() else
'cpu')

1092
1093     self.mutation_rate = initial_mutation_rate
1094     self.final_mutation_rate = final_mutation_rate
1095     self.mutation_decay = mutation_decay
1096     self.mutation_strength = mutation_strength
1097
1098     self.population = VectorizedNeuralPopulation(
1099         n_agents = n_population,
1100         n_rays = n_rays,
1101         n_system_obs = n_system_obs,
1102         hidden_size1 = hidden_size1,
1103         hidden_size2 = hidden_size2,
1104         device = str(self.device),
1105     )
1106
1107     self.generation = 0
1108     self.best_fitness_history = []
1109     self.avg_fitness_history = []
1110
1111     # Actions
1112     def get_actions(self, observations: torch.Tensor) → torch.Tensor:
1113         """
1114         Forward pass de toute la population.
1115
1116         Args:
1117             observations : Tensor (N, n_rays + 1 + n_system_obs)
1118
1119         Returns:
1120             actions : Tensor (N, 4) [steering, throttle, pit_tire, pit_full]
1121         """
1122         with torch.no_grad():
1123             return self.population.forward_vectorized(observations)
1124
1125     # Sélection / reproduction
1126     def select_top_percent(
1127         self,
1128         fitness_scores : torch.Tensor,
1129         top_percent : float = 0.1, # Le top 10%
1130     ):
1131         n_keep = max(2, int(self.n_population * top_percent))
1132         sorted_idx = torch.argsort(fitness_scores, descending=True)
1133         top_idx = sorted_idx[:n_keep]
1134         elite_genomes = self.population.genomes[top_idx].clone()
1135
1136         print(
1137             f"Sélection : top {n_keep} agents "

```

```

1138         f"(fitness max : {fitness_scores[top_idx[0]].item():.2f})\n"
1139     )
1140     return elite_genomes, top_idx
1141
1142     def reproduce(self, elite_genomes: torch.Tensor):
1143         """Crossover + mutation entièrement vectorisés - zéro boucle
Python."""
1144         n_elite      = len(elite_genomes)
1145         genome_size  = elite_genomes.shape[1]
1146         device       = self.device
1147
1148         print(f"Reproduction : {n_elite} parents → {self.n_population}
enfants\n")
1149
1150         new_genomes = torch.empty((self.n_population, genome_size),
device=device)
1151         new_genomes[:n_elite] = elite_genomes    # élitisme
1152
1153         n_children = self.n_population - n_elite
1154         if n_children > 0:
1155             idx1 = torch.randint(n_elite, (n_children,), device=device)
1156             idx2 = torch.randint(n_elite, (n_children,), device=device)
1157             p1    = elite_genomes[idx1]
1158             p2    = elite_genomes[idx2]
1159
1160             # Crossover uniforme
1161             mask    = torch.rand(n_children, genome_size, device=device) <
0.5
1162             children = torch.where(mask, p1, p2)
1163
1164             # Mutation
1165             mut_mask = torch.rand_like(children) < self.mutation_rate
1166             noise     = (torch.rand_like(children) * 2.0 - 1.0) *
self.mutation_strength
1167             children += noise * mut_mask.float()
1168
1169             new_genomes[n_elite:] = children
1170
1171             self.population.genomes = new_genomes
1172
1173         def evolve(self, fitness_scores: torch.Tensor) → dict:
1174             elite_genomes, _ = self.select_top_percent(fitness_scores,
top_percent=0.1)
1175             self.reproduce(elite_genomes)
1176
1177             self.mutation_rate = max(
1178                 self.final_mutation_rate,
1179                 self.mutation_rate * self.mutation_decay
1180             )
1181             self.generation += 1
1182

```

```

1183     best = fitness_scores.max().item()
1184     avg = fitness_scores.mean().item()
1185     self.best_fitness_history.append(best)
1186     self.avg_fitness_history.append(avg)
1187
1188     return {
1189         'generation' : self.generation,
1190         'best_fitness': best,
1191         'avg_fitness' : avg,
1192         'mutation_rate': self.mutation_rate,
1193         'elite_size' : int(self.n_population * 0.1),
1194     }
1195
1196     # Sauvegarde / chargement
1197     def save_best_agent(self, filepath: str, fitness_scores: torch.Tensor):
1198         best_idx = torch.argmax(fitness_scores).item()
1199         best_agent = self.population.get_agent(best_idx)
1200         best_agent.save_to_file(filepath)
1201         print(
1202             f"Meilleur agent sauvegardé : {filepath} "
1203             f"(fitness : {fitness_scores[best_idx].item():.2f})\n"
1204         )
1205
1206     def save_population(self, filepath: str):
1207         torch.save({
1208             'genomes'           : self.population.genomes,
1209             'generation'        : self.generation,
1210             'mutation_rate'     : self.mutation_rate,
1211             'best_fitness_history': self.best_fitness_history,
1212             'avg_fitness_history': self.avg_fitness_history,
1213             'n_rays'            : self.n_rays,
1214             'n_system_obs'      : self.n_system_obs,
1215             'hidden_size1'      : self.hidden_size1,
1216             'hidden_size2'      : self.hidden_size2,
1217         }, filepath)
1218         print(f"Population sauvegardée : {filepath}")
1219
1220     def load_population_from_file(self, filepath: str):
1221         ckpt = torch.load(filepath, map_location=self.device)
1222         if 'genomes' not in ckpt:
1223             print("Format invalide (clé 'genomes' manquante)")
1224             return
1225
1226         self.population.genomes = ckpt['genomes'].to(self.device)
1227         self.generation         = ckpt.get('generation', 0)
1228         self.mutation_rate      = ckpt.get('mutation_rate',
self.mutation_rate)
1229         self.n_system_obs       = ckpt.get('n_system_obs', N_SYSTEM_OBS)
1230         self.hidden_size1       = ckpt.get('hidden_size1', 32)
1231         self.hidden_size2       = ckpt.get('hidden_size2', 16)
1232         print(f"Population chargée : génération {self.generation}")

```

```

1233
1234 def get_statistics(self) → dict:
1235     return {
1236         'generation' : self.generation,
1237         'best_fitness_history': self.best_fitness_history,
1238         'avg_fitness_history' : self.avg_fitness_history,
1239         'current_mutation_rate': self.mutation_rate,
1240     }
1241
1242 def reset_population(self):
1243     self.population = VectorizedNeuralPopulation(
1244         n_agents      = self.n_population,
1245         n_rays        = self.n_rays,
1246         n_system_obs  = self.n_system_obs,
1247         hidden_size1  = self.hidden_size1,
1248         hidden_size2  = self.hidden_size2,
1249         device        = str(self.device),
1250     )
1251     self.generation    = 0
1252     self.mutation_rate = 0.3
1253     print("Population réinitialisée")
1254
1255
1256 class TrainingLoop:
1257     """
1258     Boucle d'entraînement avec curriculum learning en 2 phases.
1259
1260     Phase 1 - Conduite pure
1261         Les sorties pit_tire et pit_full sont masquées à 0.0 avant d'être
1262         envoyées à l'environnement. L'agent n'a aucun moyen de déclencher
1263         un pit stop, il apprend uniquement à conduire vite.
1264
1265     Phase 2 - Stratégie débloquée
1266         Déclenchée dès que l'un de ces critères est atteint :
1267         - génération ≥ phase2_gen          (critère temporel)
1268         - avg_fitness ≥ phase2_fitness      (critère qualitatif)
1269         Les pit stops deviennent accessibles, mais uniquement si la voiture
1270         est dans la zone de pit (env.in_pit_zone - défini dans
1271         neuronal_env_physic).
1272
1273         Paramètres de terminaison de génération (priorité dans l'ordre) :
1274         max_laps → s'arrête dès qu'une voiture boucle N tours
1275         max_steps → fallback si max_laps non défini
1276         Si aucun → tourne jusqu'à ce que toutes les voitures soient mortes
1277     """
1278
1279 def __init__(
1280     self,
1281     env,
1282     population_manager,
1283     fitness_tracker,

```

```

1283     frequency_show: int = 0,
1284     walls: list = None,
1285     all_configs: list = None,
1286     max_laps: int | None = None,
1287     max_steps: int | None = None,
1288     phase2_gen: int | None = 10,
1289     phase2_fitness: float | None = None,
1290 ):
1291     self.env = env
1292     self.pop_manager = population_manager
1293     self.fitness_tracker = fitness_tracker
1294     self.frequency_show = frequency_show
1295     self.walls = walls
1296     self.renderer = VectorizedRenderer(show_dead=True) if frequency_show
1297     ≠ 0 else None
1298     # Multi-circuit
1299     self.all_configs = all_configs if all_configs and len(all_configs) > 1
1300 else None
1301     self._circuit_idx = 0
1302     # Conditions d'arrêt de génération
1303     self.max_laps = max_laps
1304     self.max_steps = max_steps
1305     # Curriculum learning
1306     self.phase = 1 # démarre toujours en phase 1
1307     self.phase2_gen = phase2_gen # génération de déclenchement (None =
1308 désactivé)
1309     self.phase2_fitness = phase2_fitness # seuil fitness (None =
1310 désactivé)
1311     # Curriculum learning - masquage des actions
1312     def _apply_action_mask(self, actions: torch.Tensor) → torch.Tensor:
1313         """
1314         Applique le masque de curriculum sur les sorties pit.
1315
1316         Phase 1 : pit_tire et pit_full → 0.0 pour tout le monde
1317         Phase 2 : pit_tire et pit_full → 0.0 uniquement pour les voitures
1318 hors pit zone
1319
1320         Args:
1321             actions : Tensor (N, 4) - sortie brute de forward_vectorized
1322
1323         Returns:
1324             actions : Tensor (N, 4) - actions filtrées
1325         """
1326         if self.phase == 1:
1327             # Masque total : aucun pit possible
1328             actions[:, 2:4] = 0.0

```

```

1329         elif self.phase == 2:
1330             # Masque partiel : pit interdit hors zone
1331             # env.in_pit_zone est un Tensor (N,) bool mis à jour dans
neural_env_physic.step()
1332             if hasattr(self.env, 'in_pit_zone'):
1333                 out_of_pit = ~self.env.in_pit_zone           # (N,) bool
1334                 actions[out_of_pit, 2:4] = 0.0
1335             else:
1336                 # Sécurité : si l'env ne définit pas in_pit_zone, on masque
tout
1337                 actions[:, 2:4] = 0.0
1338
1339         return actions
1340
1341     def _check_phase_transition(self, stats: dict):
1342         """
1343         Vérifie si les critères de passage en phase 2 sont atteints.
1344         Affiche un message une seule fois au moment de la transition.
1345         """
1346         if self.phase != 1:
1347             return # déjà en phase 2, rien à faire
1348
1349         triggered = False
1350         reason = ""
1351
1352         if self.phase2_gen is not None:
1353             if self.pop_manager.generation ≥ self.phase2_gen:
1354                 triggered = True
1355                 reason = f"génération {self.pop_manager.generation} ≥
{self.phase2_gen}"
1356
1357         if self.phase2_fitness is not None and not triggered:
1358             if stats.get('avg_fitness', 0.0) ≥ self.phase2_fitness:
1359                 triggered = True
1360                 reason = (
1361                     f"avg_fitness {stats['avg_fitness']:.2f} "
1362                     f"≥ seuil {self.phase2_fitness:.2f}"
1363                 )
1364
1365         if triggered:
1366             self.phase = 2
1367             print(f"\n{'='*60}")
1368             print(f"  PHASE 2 DÉBLOQUÉE - Pit stops activés ({reason})")
1369             print(f"{'='*60}\n")
1370
1371         # Rotation de circuit
1372     def _rotate_circuit(self):
1373         cfg = self.all_configs[self._circuit_idx]
1374         self._circuit_idx = (self._circuit_idx + 1) % len(self.all_configs)
1375
1376         self.env = cfg["env"]

```

```

1377     self.walls = cfg["walls"]
1378
1379     new_checkpoints = cfg["checkpoints"]
1380     n_new = len(new_checkpoints)
1381
1382     self.fitness_tracker.checkpoints = torch.tensor(
1383         new_checkpoints, dtype=torch.float32,
1384         device=self.fitness_tracker.device
1385     )
1386     self.fitness_tracker.n_checkpoints = n_new
1387     self.fitness_tracker.spawn_point = (
1388         self.env.spawn_x, self.env.spawn_y, self.env.spawn_angle
1389     )
1390     self.fitness_tracker._compute_finish_line(
1391         (self.env.spawn_x, self.env.spawn_y, self.env.spawn_angle),
1392         cfg["walls"]
1393     )
1394     self.fitness_tracker.checkpoint_status = torch.zeros(
1395         (self.fitness_tracker.n_cars, n_new),
1396         dtype=torch.bool, device=self.fitness_tracker.device
1397     )
1398     self.fitness_tracker.treshold = self.env.track_width
1399
1400     print(f"  Circuit : {cfg['name']} (difficulté {cfg['difficulty']})")
1401
1402     # Condition d'arrêt de génération (identique à v1)
1403     def _is_generation_done(self, step: int) → bool:
1404         if not self.env.alive.any():
1405             return True
1406         if self.max_laps is not None:
1407             if bool((self.fitness_tracker.laps_completed ≥
self.max_laps).any()):
1408                 return True
1409         if self.max_steps is not None:
1410             if step ≥ self.max_steps:
1411                 return True
1412         return False
1413
1414     # Boucle de génération
1415     def run_generation(self, render: bool = False, generation: int = 0) →
dict | None:
1416         observations = self.env.reset()
1417         self.fitness_tracker.reset()
1418
1419         step = 0
1420         while not self._is_generation_done(step):
1421
1422             # Forward pass réseau
1423             actions = self.pop_manager.get_actions(observations)
1424

```

```

1425         # -- Curriculum : masquage des pit stops selon la phase -----
1426         --
1427         actions = self._apply_action_mask(actions)
1428         # Step environnement
1429         observations, rewards, dones = self.env.step(actions)
1430
1431         self.fitness_tracker.update(
1432             positions = self.env.pos,
1433             speeds = self.env.speed,
1434             alive_mask = self.env.alive,
1435         )
1436
1437         if render:
1438             render_data = self.env.get_render_data()
1439             still_running = self.renderer.render_step(generation,
render_data, self.walls, self.fitness_tracker.get_render_finish_line())
1440             if not still_running:
1441                 return None
1442
1443             step += 1
1444
1445             best_laps = int(self.fitness_tracker.laps_completed.max().item())
1446             if best_laps > 0:
1447                 print(f"Meilleur tour bouclé : {best_laps} (sur {step} steps)")
1448             else:
1449                 print(f"Génération terminée après {step} steps (aucun tour
complet)")
1450
1451             fitness_scores = self.fitness_tracker.compute_fitness()
1452             stats = self.pop_manager.evolve(fitness_scores)
1453             stats.update(self.fitness_tracker.get_statistics())
1454
1455             # -- Vérification transition de phase -----
1456             -
1457             self._check_phase_transition(stats)
1458
1459             return stats
1460
1461             # Boucle d'entraînement principale
1462             def train(
1463                 self,
1464                 n_generations : int = 100,
1465                 save_every : int = 10,
1466                 save_path : str = "checkpoints",
1467             ):
1468                 os.makedirs(save_path, exist_ok=True)
1469                 print(
1470                     f"Début : {n_generations} générations | "
1471                     f"Phase 2 : gen≥{self.phase2_gen} ou
fitness≥{self.phase2_fitness}\n"

```



```

1471         + "=" * 100
1472     )
1473
1474     for gen in range(n_generations):
1475         print(f"\nGÉNÉRATION {gen + 1}/{n_generations} [Phase
1476         {self.phase}]")
1477
1478         if self.all_configs is not None:
1479             self._rotate_circuit()
1480
1481         render = (
1482             self.frequency_show != 0
1483             and (gen + 1) % self.frequency_show == 0
1484         )
1485         stats = self.run_generation(render=render, generation=gen)
1486
1487         if stats is None:
1488             print("Fenêtre fermée, arrêt de l'entraînement.")
1489             return
1490
1491         print(
1492             f"Best={stats['best_fitness']:.2f} | "
1493             f"Avg={stats['avg_fitness']:.2f} | "
1494             f"Mutation={stats['mutation_rate']:.1%} | "
1495             f"Meilleurs tours={stats['max_laps']}"
1496         )
1497         print("-" * 60)
1498
1499         if (gen + 1) % save_every == 0:
1500             self.pop_manager.save_population(
1501                 os.path.join(save_path, f"gen_{gen + 1}.pt")
1502             )
1503
1504
1505
1506 """
1507 circuits_loader.py
1508 =====
1509 Identique à circuits_loader.py avec deux différences :
1510
1511 1. Instancie VectorizedCarEnv depuis envs.neuronal_env_physic
1512    (physique réaliste, systèmes pneus/fuel/pluie, collision GPU)
1513
1514 2. Accepte rain_mode, initial_rain, initial_compound
1515    pour configurer les systèmes dès la construction de l'env.
1516
1517 Contrat attendu dans chaque circuit_*.py:
1518     get_name()          → str
1519     get_difficulty()    → int
1520     get_walls()         → list of ((x1,y1),(x2,y2))

```

```

1521     get_spawn()          → (x, y, theta)
1522     get_width()          → float
1523     get_checkpoints()    → list of (x, y)
1524     get_track_geo()      → RectangularTrack | AngularTrack
1525
1526 Usage :
1527     from tracks.circuits_loader import load_circuits
1528     """
1529
1530     import importlib.util
1531     import sys
1532     from pathlib import Path
1533     from envs.neuronal_env_physic import VectorizedCarEnv
1534     from physics.tires_system import MEDIUM
1535
1536     _REQUIRED = (
1537         "get_name", "get_difficulty", "get_walls",
1538         "get_spawn", "get_width", "get_checkpoints", "get_track_geo"
1539     )
1540
1541
1542     # -- Chargement dynamique -----
1543
1544     def _load_module(path: Path):
1545         module_name = f"_circuit_dyn_{path.stem}"
1546         if module_name in sys.modules:
1547             return sys.modules[module_name]
1548         spec = importlib.util.spec_from_file_location(module_name, path)
1549         module = importlib.util.module_from_spec(spec)
1550         try:
1551             spec.loader.exec_module(module)
1552         except Exception as e:
1553             print(f"[circuits_loader] Impossible de charger {path.name} : {e}")
1554             return None
1555         sys.modules[module_name] = module
1556         return module
1557
1558
1559     def _validate(module, path: Path) → bool:
1560         missing = [fn for fn in _REQUIRED if not hasattr(module, fn)]
1561         if missing:
1562             print(
1563                 f"[circuits_loader] {path.name} ignoré - "
1564                 f"fonctions manquantes : {'', '.join(missing)}"
1565             )
1566             return False
1567         return True
1568
1569
1570     # -- Construction de l'env (v2 : paramètres systèmes ajoutés) -----

```

```
1571
1572 def _build_config(
1573     module,
1574     n_cars          : int,
1575     n_rays          : int,
1576     device          : str,
1577     rain_mode       : str,
1578     initial_rain    : float,
1579     initial_compound : int,
1580 ) → dict | None:
1581     name = module.get_name()
1582     try:
1583         walls      = module.get_walls()
1584         spawn      = module.get_spawn()
1585         track_width = module.get_width()
1586         checkpoints = module.get_checkpoints()
1587         track       = module.get_track_geo()
1588     except Exception as e:
1589         print(f"[circuits_loader] Erreur lecture '{name}' : {e}")
1590         return None
1591
1592     try:
1593         env = VectorizedCarEnv(
1594             spawn_point    = spawn,
1595             walls          = walls,
1596             track          = track,
1597             track_width    = track_width,
1598             n_cars         = n_cars,
1599             n_rays         = n_rays,
1600             device         = device,
1601             rain_mode      = rain_mode,
1602             initial_rain   = initial_rain,
1603             initial_compound = initial_compound,
1604         )
1605     except Exception as e:
1606         print(f"[circuits_loader] VectorizedCarEnv a échoué pour '{name}' : {e}")
1607         return None
1608
1609     return {
1610         "name"          : name,
1611         "difficulty"    : module.get_difficulty(),
1612         "env"           : env,
1613         "checkpoints"   : checkpoints,
1614         "walls"         : walls,
1615     }
1616
1617
1618 # -- Interface publique -----
1619
```

```

1620 def load_circuits(
1621     tracks_dir      : str | Path = "tracks",
1622     n                : int | None = None,
1623     names            : list | None = None,
1624     n_cars            : int   = 1000,
1625     n_rays            : int   = 9,
1626     device           : str    = 'cuda',
1627     rain_mode         : str    = 'fixed',
1628     initial_rain      : float  = 0.0,
1629     initial_compound : int    = MEDIUM,
1630 ) → list:
1631     """
1632     Charge tous les circuits valides et retourne une liste de configs.
1633     """
1634     tracks_path = Path(tracks_dir)
1635     circuit_files = sorted(tracks_path.glob("circuit_*.py"))
1636
1637     if not circuit_files:
1638         raise FileNotFoundError(
1639             f"Aucun fichier circuit_*.py trouvé dans {tracks_path.resolve()}"
1640         )
1641
1642     modules = []
1643     for path in circuit_files:
1644         module = _load_module(path)
1645         if module and _validate(module, path):
1646             modules.append(module)
1647
1648     if not modules:
1649         raise RuntimeError("Aucun circuit valide n'a pu être chargé.")
1650
1651     modules.sort(key=lambda m: m.get_difficulty())
1652
1653     if names is not None:
1654         name_set = set(names)
1655         modules = [m for m in modules if m.get_name() in name_set]
1656         not_found = name_set - {m.get_name() for m in modules}
1657         if not_found:
1658             print(f"Circuits introuvables : {not_found}")
1659     elif n is not None:
1660         modules = modules[:n]
1661
1662     configs = []
1663     for module in modules:
1664         cfg = _build_config(
1665             module, n_cars, n_rays, device,
1666             rain_mode, initial_rain, initial_compound
1667         )
1668         if cfg is not None:
1669             configs.append(cfg)
1670

```

```
1671     if not configs:
1672         raise RuntimeError("Aucun environnement n'a pu être construit.")
1673
1674     print(
1675         f"Circuit loader: {len(configs)} circuit(s) prêt(s) : "
1676         + ", ".join(f"{c['name']}(diff={c['difficulty']})" for c in configs)
1677     )
1678     return configs
1679
1680
1681
1682
1683 """w_track.py
1684
1685 Circuit w, imaginaire mais intéressant, généré par spline Catmull-Rom.
1686 Difficulté 4 – circuit complexe avec courbes enchaînées, virage serrés.
1687 """
1688
1689 from tracks.Catmull_Rom_geometry import catmull_rom_spline
1690 from tracks.track_geometry import generate_walls, compute_centerline
1691 from tracks.track_geometry import AngularTrack
1692
1693 def get_name():
1694     return "w_track"
1695
1696 def get_difficulty():
1697     return 4
1698
1699 def get_spawn():
1700     return 9 * 31.400, 9 * 27.457, 0.0
1701
1702 def _get_centerline():
1703     line = [
1704         (9 * 31.400, 9 * 27.457),
1705         (9 * 44.000, 9 * 29.143),
1706         (9 * 48.500, 9 * 32.107),
1707         (9 * 54.400, 9 * 32.814),
1708         (9 * 58.400, 9 * 30.600),
1709         (9 * 62.700, 9 * 27.764),
1710         (9 * 68.800, 9 * 27.557),
1711         (9 * 71.700, 9 * 29.336),
1712         (9 * 74.000, 9 * 32.714),
1713         (9 * 72.100, 9 * 36.150),
1714         (9 * 69.000, 9 * 38.786),
1715         (9 * 64.600, 9 * 40.757),
1716         (9 * 59.000, 9 * 42.929),
1717         (9 * 52.000, 9 * 43.643),
1718         (9 * 45.000, 9 * 43.286),
1719         (9 * 42.000, 9 * 41.929),
1720         (9 * 38.800, 9 * 39.843),
1721         (9 * 33.800, 9 * 37.557),
```

```
1722     (9 * 29.800, 9 * 37.129),
1723     (9 * 25.200, 9 * 38.300),
1724     (9 * 19.600, 9 * 38.757),
1725     (9 * 14.800, 9 * 35.629),
1726     (9 * 12.000, 9 * 31.000),
1727     (9 * 7.5000, 9 * 24.250),
1728     (9 * 7.4000, 9 * 21.457),
1729     (9 * 11.800, 9 * 18.986),
1730     (9 * 19.200, 9 * 21.514),
1731     (9 * 24.400, 9 * 26.314)
1732
1733 ]
1734 return line
1735
1736 def _build_walls_width():
1737     """Construction unique – partagée entre get_walls"""
1738     control_points = _get_centerline()
1739     centerline = catmull_rom_spline(control_points)
1740     outer, inner, width = generate_walls(centerline)
1741     return outer, inner, width
1742
1743 def get_width():
1744     _, _, width = _build_walls_width()
1745     return width
1746
1747 def get_walls():
1748     outer, inner, _ = _build_walls_width()
1749     walls = [(inner[i], inner[i+1]) for i in range(len(inner) - 1)]
1750     walls += [(outer[i], outer[i+1]) for i in range(len(outer) - 1)]
1751     return walls
1752
1753 def get_checkpoints():
1754     outer, inner, _ = _build_walls_width()
1755     centerline = compute_centerline(outer, inner)
1756     return centerline[::20]
1757
1758 def get_track_geo():
1759     outer, inner, _ = _build_walls_width()
1760     outer, inner, _ = _build_walls_width()
1761
1762     # outer et inner sont inversés sur ce circuit – on les swap
1763     outer_closed = inner + [inner[0]]
1764     inner_closed = outer + [outer[0]]
1765
1766     return AngularTrack(outer_closed, inner_closed)
1767
1768
1769
1770
1771
1772
```

```
1773
1774 import numpy as np
1775
1776 def catmull_rom_segment(p0, p1, p2, p3, n_points=20):
1777     """
1778     Génère n_points sur la spline Catmull-Rom entre p1 et p2
1779     """
1780     p0, p1, p2, p3 = map(np.array, (p0, p1, p2, p3))
1781     points = []
1782
1783     for t in np.linspace(0, 1, n_points):
1784         t2 = t * t
1785         t3 = t2 * t
1786
1787         point = 0.5 * (
1788             (2 * p1)
1789             + (-p0 + p2) * t
1790             + (2*p0 - 5*p1 + 4*p2 - p3) * t2
1791             + (-p0 + 3*p1 - 3*p2 + p3) * t3
1792         )
1793         points.append(tuple(point))
1794
1795     return points
1796
1797 def catmull_rom_spline(points, n_points_per_segment=20, closed=True):
1798     """
1799     Génère une spline Catmull-Rom complète
1800     """
1801     if closed:
1802         pts = [points[-1]] + points + [points[0], points[1]]
1803     else:
1804         pts = [points[0]] + points + [points[-1]]
1805
1806     curve = []
1807
1808     for i in range(1, len(pts) - 2):
1809         seg = catmull_rom_segment(
1810             pts[i-1],
1811             pts[i],
1812             pts[i+1],
1813             pts[i+2],
1814             n_points_per_segment
1815         )
1816         curve.extend(seg)
1817
1818     return curve
1819
1820
1821 import numpy as np
1822 import matplotlib.pyplot as plt
1823
```

```

1824
1825 def compute_tangents(centerline: np.ndarray) → np.ndarray:
1826     """
1827     Calcule les vecteurs tangents par différences centrées.
1828     """
1829     N = len(centerline)
1830     tangents = np.zeros_like(centerline)
1831
1832     for i in range(N):
1833         p_prev = centerline[i - 1]
1834         p_next = centerline[(i + 1) % N] # périodique (circuit fermé)
1835         tangents[i] = p_next - p_prev
1836
1837     return tangents
1838
1839
1840 def compute_normals(tangents) → np.ndarray:
1841     """
1842     Calcule les normales unitaires à partir des tangentes.
1843     """
1844     normals = np.zeros_like(tangents)
1845
1846     for i, t in enumerate(tangents):
1847         norm = np.linalg.norm(t)
1848         if norm == 0:
1849             raise ValueError(f"Tangente nulle au point {i}")
1850         normals[i] = np.array([-t[1], t[0]]) / norm
1851
1852     return normals
1853
1854
1855 def generate_walls(centerline: np.ndarray, width: float):
1856     """
1857     Génère les murs gauche et droit à partir de la ligne centrale.
1858     """
1859     tangents = compute_tangents(centerline)
1860     normals = compute_normals(tangents)
1861
1862     left_wall = centerline + width * normals
1863     right_wall = centerline - width * normals
1864
1865     return left_wall, right_wall
1866
1867
1868 def plot_track(centerline, left_wall, right_wall):
1869     """
1870     Visualisation de contrôle géométrique.
1871     """
1872     plt.figure(figsize=(8, 8))
1873     plt.plot(centerline[:, 0], centerline[:, 1], 'k--', label="Ligne
centrale")

```



```

1874 plt.plot(left_wall[:, 0], left_wall[:, 1], 'r', label="Mur gauche")
1875 plt.plot(right_wall[:, 0], right_wall[:, 1], 'b', label="Mur droit")
1876 plt.axis('equal')
1877 plt.legend()
1878 plt.title("Génération des murs du circuit")
1879 plt.show()
1880
1881 """
1882 adaptation_wtrack.py – Fine-tuning des 3 agents pré-entraînés sur W-track
1883 =====
1884
1885 Sauvegarde le résultat dans le même dossier que le checkpoint source,
1886 sous le nom adaptation_w_track.pt (même architecture que les gen_N.pt).
1887
1888 Usage :
1889     python -m simulations.adaptation_wtrack \
1890         --nascar checkpoints/nascar_.../gen_50.pt \
1891         --gt checkpoints/speed_ring_gt_.../gen_50.pt \
1892         --multi checkpoints/nascar_rectangle_speed_ring_gt_.../gen_50.pt \
1893         --n_adapt_gen 10
1894 """
1895
1896 import os
1897 os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
1898
1899 import torch
1900 import argparse
1901 from pathlib import Path
1902
1903 from learnings.genetic_algorithm.fitness_tracker import FitnessTracker
1904 from learnings.genetic_algorithm.pop_manager_physic import PopulationManager,
1905 TrainingLoop
1906 from tracks.circuits_loader import load_circuits
1907
1908 # -----
1909 # Arguments
1910 # -----
1911
1912 def parse_args():
1913     parser = argparse.ArgumentParser(
1914         description="Adaptation des agents pré-entraînés sur W-track"
1915     )
1916
1917     parser.add_argument('--nascar', type=str, required=True)
1918     parser.add_argument('--gt', type=str, required=True)
1919     parser.add_argument('--multi', type=str, required=True)
1920
1921     parser.add_argument('--n_adapt_gen', type=int, default=10)
1922     parser.add_argument('--population', type=int, default=1000)
1923     parser.add_argument('--n_rays', type=int, default=9)

```

```

1924     parser.add_argument('--nb_steps', type=int, default=2000)
1925     parser.add_argument('--nb_laps', type=int, default=1)
1926     parser.add_argument('--device', type=str, default='cuda', choices=
1927     ['cuda', 'cpu'])
1928
1929     parser.add_argument('--mutation_start', type=float, default=0.05)
1930     parser.add_argument('--mutation_end', type=float, default=0.005)
1931     parser.add_argument('--mutation_decay', type=float, default=0.90)
1932
1933     return parser.parse_args()
1934
1935
1936 # -----
1937 # Adaptation d'un seul agent
1938 # -----
1939
1940 def run_adaptation(label, checkpoint_path, env, checkpoints, walls, args):
1941     """
1942     Charge un checkpoint, lance N générations sur W-track,
1943     sauvegarde adaptation_w_track.pt dans le même dossier que le checkpoint.
1944     Retourne le chemin du fichier produit.
1945     """
1946     print(f"\n{'='*70}")
1947     print(f"  ADAPTATION : {label}")
1948     print(f"  Checkpoint : {checkpoint_path}")
1949     print(f"{'='*70}")
1950
1951     # -- Population -----
1952
1953     pop_manager = PopulationManager(
1954         n_population      = args.population,
1955         n_rays            = args.n_rays,
1956         initial_mutation_rate = args.mutation_start,
1957         final_mutation_rate  = args.mutation_end,
1958         mutation_decay      = args.mutation_decay,
1959         device            = args.device,
1960     )
1961     pop_manager.load_population_from_file(checkpoint_path)
1962     pop_manager.generation = 0
1963
1964     # -- Fitness tracker -----
1965
1966     fitness_tracker = FitnessTracker(
1967         checkpoints = checkpoints,
1968         spawn_point = (env.spawn_x, env.spawn_y, env.spawn_angle),
1969         n_cars      = args.population,
1970         track_width = env.track_width,
1971         walls       = walls,
1972         device      = args.device,
1973     )

```

```

1972
1973 # -- Boucle d'adaptation -----
1974
1975 training_loop = TrainingLoop(
1976     env = env,
1977     population_manager = pop_manager,
1978     fitness_tracker = fitness_tracker,
1979     frequency_show = args.frequency_show,
1980     walls = walls,
1981     max_laps = args.nb_laps,
1982     max_steps = args.nb_steps,
1983     phase2_gen = None,
1984     phase2_fitness = None,
1985 )
1986
1987 for gen in range(args.n_adapt_gen):
1988     print(f"\n Génération {gen + 1}/{args.n_adapt_gen}")
1989     stats = training_loop.run_generation(generation=gen)
1990     if stats is None:
1991         break
1992     print(
1993         f"    Best={stats['best_fitness']:.1f} | "
1994         f"Avg={stats['avg_fitness']:.1f} | "
1995         f"Mutation={stats['mutation_rate']:.2%}"
1996     )
1997
1998 # -- Sauvegarde dans le même dossier que le checkpoint source -----
1999
2000 save_path = Path(checkpoint_path).parent / "adaptation_w_track.pt"
2001 pop_manager.save_population(str(save_path))
2002 print(f"\n Sauvegardé : {save_path}")
2003 return str(save_path)
2004
2005 # -----
2006 # Main
2007 # -----
2008
2009 def main():
2010     args = parse_args()
2011
2012     if args.device == 'cuda' and not torch.cuda.is_available():
2013         print("CUDA non disponible, passage en CPU")
2014         args.device = 'cpu'
2015
2016 # -- W-track chargé une seule fois, partagé entre les 3 agents -----
2017 print("\nChargement du circuit W-track ... ")
2018 configs = load_circuits(
2019     names = ['w_track'],
2020     n_cars = args.population,
2021     n_rays = args.n_rays,

```

```
2021         device = args.device,
2022     )
2023     cfg          = configs[0]
2024     env          = cfg["env"]
2025     checkpoints  = cfg["checkpoints"]
2026     walls        = cfg["walls"]
2027     print(f"  Checkpoints : {len(checkpoints)}")
2028
2029     # -- 3 adaptations séquentielles -----
2030     -
2031     agents = [
2032         ("NASCAR",      args.nascar),
2033         ("Speed Ring GT", args.gt),
2034         ("Multi-circuit", args.multi),
2035     ]
2036     result_paths = {}
2037     for label, ckpt in agents:
2038         result_paths[label] = run_adaptation(
2039             label, ckpt, env, checkpoints, walls, args
2040         )
2041
2042     # -- Commande show_comparison -----
2043     -
2044     print(f"\n{'='*70}")
2045     print("  ADAPTATION TERMINÉE – commande pour la comparaison :")
2046     print(f"{'='*70}")
2047     print(
2048         f"\npython -m simulations.show_comparison \\\n"
2049         f"    --nascar  {result_paths['NASCAR']} \\\n"
2050         f"    --gt      {result_paths['Speed Ring GT']} \\\n"
2051         f"    --multi   {result_paths['Multi-circuit']}"
2052     )
2053
2054     if __name__ == "__main__":
2055         main()
2056
2057
2058
```